



Antonio Piemontese <apiemonte56@gmail.com>

Full-model Fine-tuning vs. LoRA vs. RAG

1 messaggio

Daily Dose of Data Science <avichawla@substack.com>

1 marzo 2024 alle ore 20:21

Rispondi a: Daily Dose of Data Science

<reply+2co5cn&3hg5ec&&9f7788c2c4a31379acd0238d59cee1ee38f6a02cd0126074e35cbe7a01752f52@mg1.substack.com>

A: apiemonte56@gmail.com

Forwarded this email? [Subscribe here](#) for more

Full-model Fine-tuning vs. LoRA vs. RAG

A visual summary

AVI CHAWLA

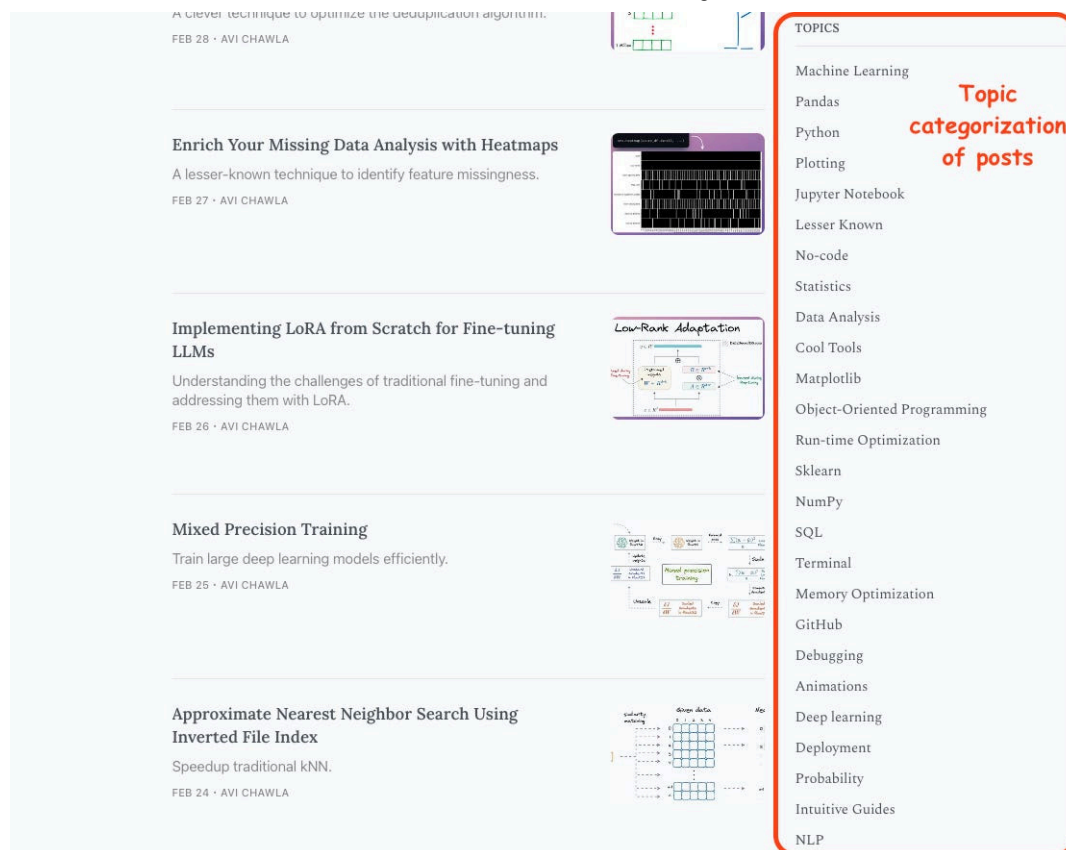
MAR 1



READ IN APP ↗

Before we begin...

If you don't know yet, the posts of this newsletter are already grouped based on topics. Go to the [home page](#) of this newsletter and find the "TOPICS" section:



You will find all the topics here.

I thought it would be good to let you know because so many of you have reached out lately as they wanted a better way to go through older posts on specific topics.

I hope this will be helpful :)

Let's get to today's post now.

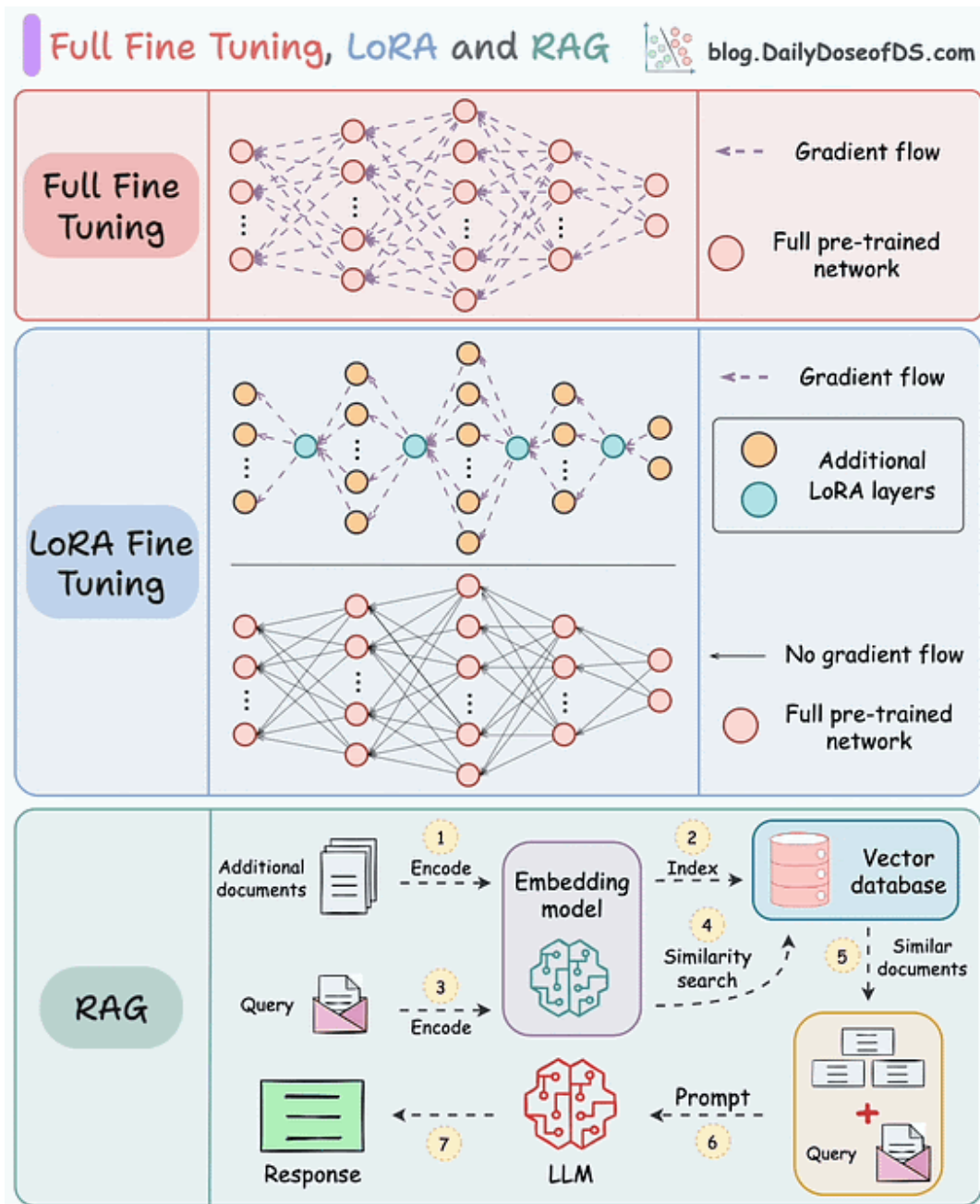
Full-model Fine-tuning vs. LoRA vs. RAG

Over the last couple of weeks, we covered several details around vector databases, fine-tuning LLMs, RAGs, and more.

If you are new here (or wish to recall), you can read this:

- [A Beginner-friendly and Comprehensive Deep Dive on Vector Databases.](#)
- [Implementing LoRA From Scratch for Fine-tuning LLMs.](#)

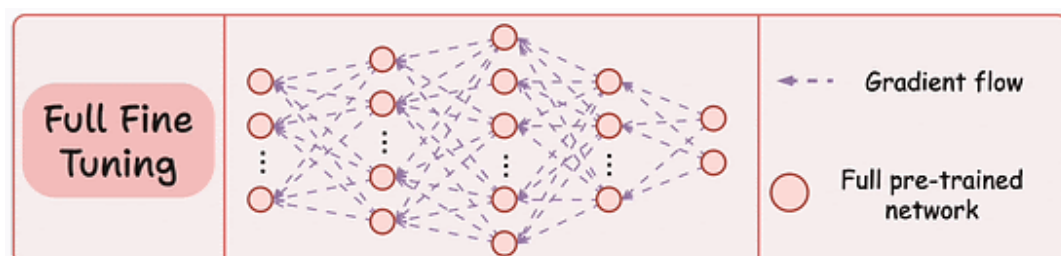
I prepared the following visual, which illustrates the “**full-model fine-tuning**,” “**fine-tuning with LoRA**,” and “**retrieval augmented generation (RAG)**.”



All three techniques are used to augment the knowledge of an existing model with additional data.

#1) Full fine-tuning

Fine-tuning means adjusting the weights of a **pre-trained model** on a new dataset for better performance.



While this fine-tuning technique has been successfully used for a long time, problems arise when we use it on much larger models — LLMs, for instance, primarily because of:

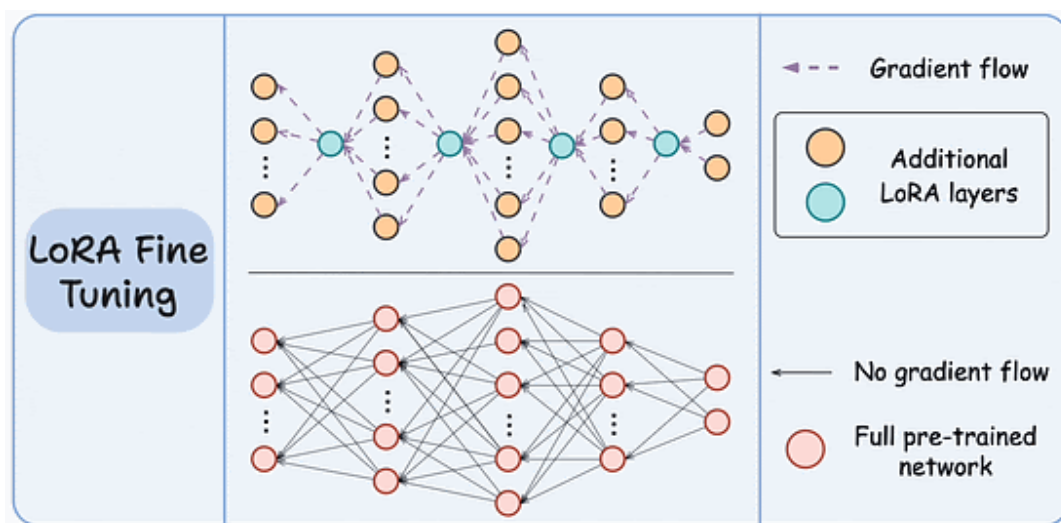
1. Their size.
2. The cost involved in fine-tuning all weights.
3. The cost involved in maintaining all large fine-tuned models.

#2) LoRA fine-tuning

LoRA fine-tuning addresses the limitations of traditional fine-tuning.

The core idea is to decompose the weight matrices (some or all) of the original model into low-rank matrices and train them instead.

For instance, in the graphic below, the bottom network represents the large pre-trained model, and the top network represents the model with LoRA layers.



The idea is to train only the LoRA network and freeze the large model.

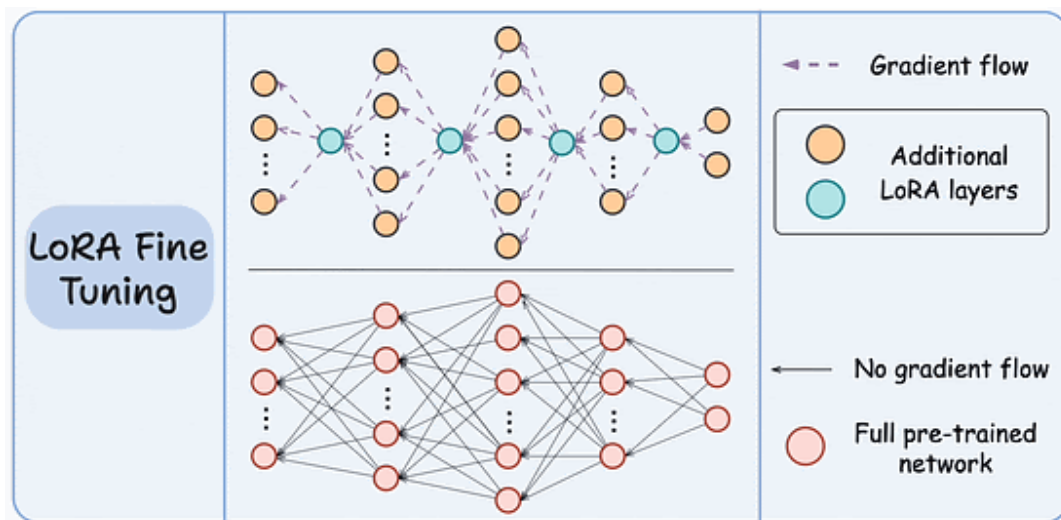
Looking at the above visual, you might think:

But the LoRA model has more neurons than the original model. How does that help?

To understand this, you must make it clear that neurons have anything to do with the memory of the network. They are just used to illustrate the dimensionality transformation from one layer to another.

It is the weight matrices (*or the connections between two layers*) that take up memory.

Thus, we must be comparing these connections instead:



Looking at the above visual, it is pretty clear that the LoRA network has relatively very few connections.

To get into more detail on:

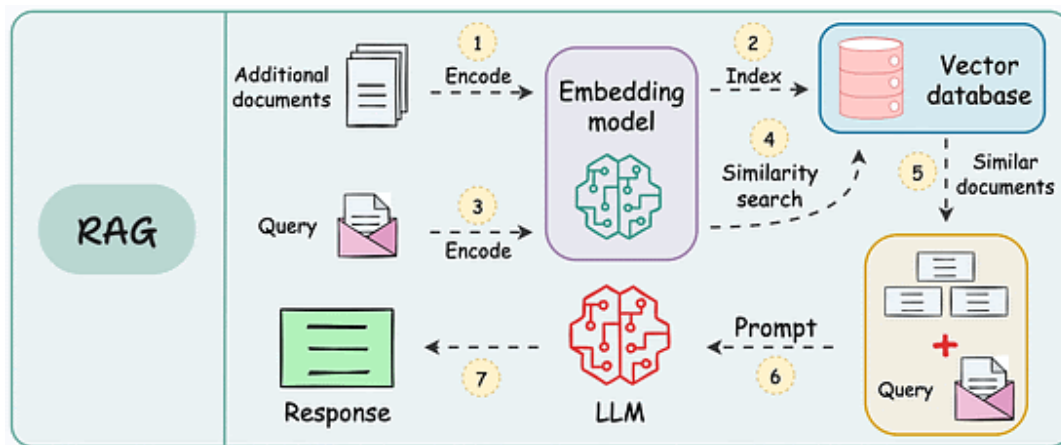
- How LoRA works?
- Why is it effective and cost savings over traditional fine-tuning?
- How to implement it from scratch?
- How to use Huggingface PEFT to fine-tune any model using LoRA?

...read this article: [Implementing LoRA From Scratch for Fine-tuning LLMs](#).

#3) RAG

Retrieval augmented generation (RAG) is another pretty cool way to augment neural networks with additional information, **without having to fine-tune the model**.

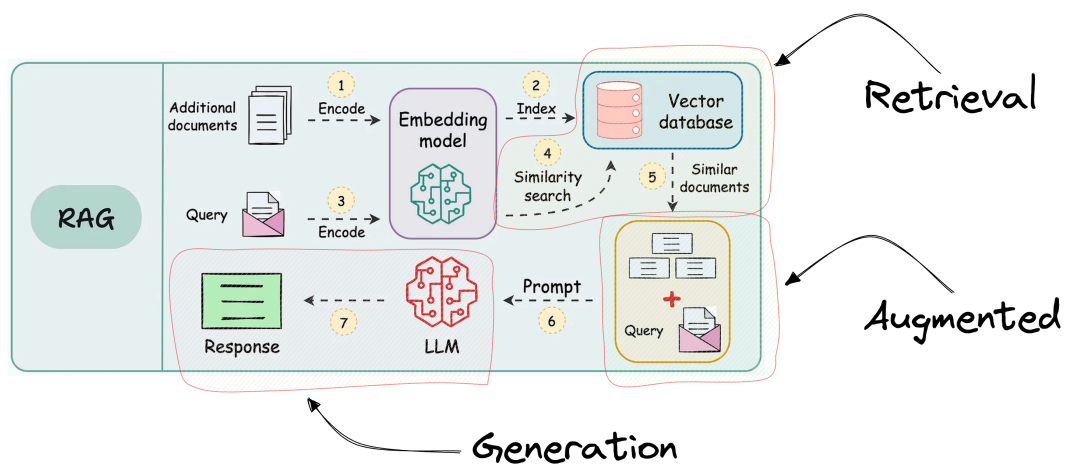
This is illustrated below:



There are 7 steps, which are also marked in the above visual:

- **Step 1-2:** Take additional data, and dump it in a vector database after embedding. (This is only done once. If the data is evolving, just keep dumping the embeddings into the vector database. There's no need to repeat this again for the entire data)
- **Step 3:** Use the same embedding model to embed the user query.
- **Step 4-5:** Find the nearest neighbors in the vector database to the embedded query.
- **Step 6-7:** Provide the original query and the retrieved documents (for more context) to the LLM to get a response.

In fact, even its name entirely justifies what we do with this technique:



- **Retrieval:** Accessing and retrieving information from a knowledge source, such as a database or memory.
- **Augmented:** Enhancing or enriching something, in this case, the text generation process, with additional information or context.

- **Generation:** The process of creating or producing something, in this context, generating text or language.

Of course, there are many problems with RAG too, such as:

- RAGs involve similarity matching between the query vector and the vectors of the additional documents. However, questions are structurally very different from answers.
- Typical RAG systems are well-suited only for lookup-based question-answering systems. For instance, we cannot build a RAG pipeline to summarize the additional data. The LLM never gets info about all the documents in its prompt because the similarity matching step only retrieves top matches.

So, it's pretty clear that RAG has both pros and cons.

- We never have to fine-tune the model, which saves a lot of computing power.
- But this also limits the applicability to specific types of systems.

Do check out these deep dives for better clarity on this topic:

- [A Beginner-friendly and Comprehensive Deep Dive on Vector Databases.](#)
- [Implementing LoRA From Scratch for Fine-tuning LLMs.](#)

👉 Over to you: Can you point out some more problems with RAGs?

👉 If you liked this post, don't forget to leave a like ❤️. It helps more people discover this newsletter on Substack and tells me that you appreciate reading these daily insights.

4-5 hours

Time spent to
research, draft,
edit and send
a newsletter
issue everyday.

2 seconds

Time spent to
like the post and
show your support

The button is located towards the bottom of this email.

Thanks for reading!

Whenever you are ready, here's one more way I can help you:

Every week, I publish 1-2 in-depth deep dives (typically 20+ mins long). Here are some of the latest ones that you will surely like:

[I want to read full articles.](#)

- [\[FREE\] A Beginner-friendly and Comprehensive Deep Dive on Vector Databases.](#)
- [Implementing LoRA From Scratch for Fine-tuning LLMs.](#)
- [You Are Probably Building Inconsistent Classification Models Without Even Realizing](#)
- [Why Sklearn's Logistic Regression Has no Learning Rate Hyperparameter?](#)
- [PyTorch Models Are Not Deployment-Friendly! Supercharge Them With TorchScript.](#)
- [DBSCAN++: The Faster and Scalable Alternative to DBSCAN Clustering.](#)
- [Federated Learning: A Critical Step Towards Privacy-Preserving Machine Learning.](#)
- [You Cannot Build Large Data Projects Until You Learn Data Version Control!](#)

To receive all full articles and support the Daily Dose of Data Science, consider subscribing:

[I want to read full articles.](#)

👉 If you love reading this newsletter, feel free to share it with friends!

Share Daily Dose of Data Science

 LIKE

 COMMENT

 RESTACK

© 2024 Avi Chawla
India
[Unsubscribe](#)

Get the app

 Start writing